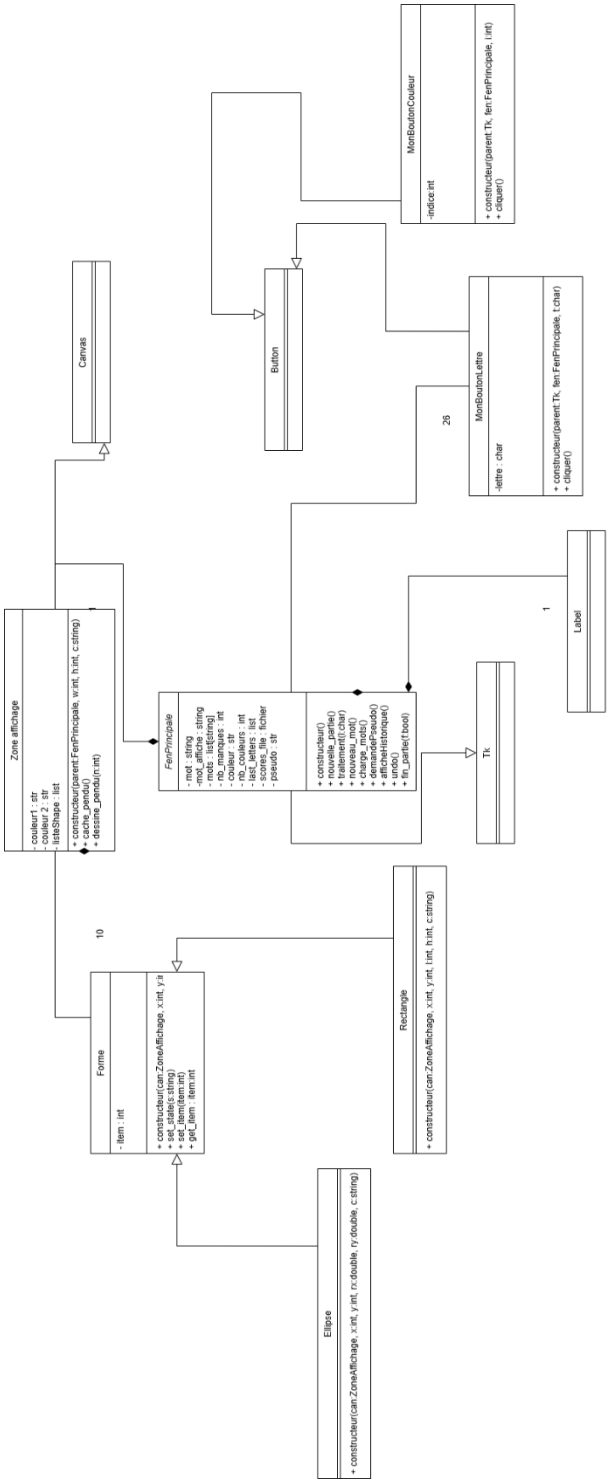
Cassette Hugo, Favre-Bulle Romain ARDABILIAN Mohsen

Rapport TD #3 - INF-TC2

Diagramme UML



Présentation du code

Couleur

Nous avons créé une classe MonBouttonCouleur sur le modèle de la classe MonBouttonLettre. On a ajouté dans les attributs de la classe un indice i, qui correspond à l'indice de la couleur en question dans la liste « self.couleurs », attribut de la fenêtre principale (il s'agit de l'ensemble des couleurs modifiables sur la fenêtre).

En fonction de l'indice sélectionné, on modifie la couleur de l'objet concerné.

```
class Mo   Button widget.
    def    Voir le problème (Alt+F8)   Correction rapide... (Ctrl+;)
        Button.__init__(self, master=parent, text="Choix couleur " + str(i+1))
        self.__i = i
        self.__fen = fen

    def cliquer(self):
        c = askcolor()
        if c[1]:
            self.__fen.couleurs[self.__i] = c[1]
            if self.__i == 0:
                self.__fen.configure(bg=self.__fen.couleurs[self.__i])
            elif self.__i == 1:
                self.__fen.zoneAffichage.configure(bg=self.__fen.couleurs[self.__i])
            elif self.__i == 2:
                for i in range(4):
                    self.__fen.zoneAffichage.listeShape[i].change_couleur(self.__fen.couleurs[self.__i])
            elif self.__i == 3:
                for i in range(4,10):
                    self.__fen.zoneAffichage.listeShape[i].change_couleur(self.__fen.couleurs[self.__i])
```

Undo

Nous avons également ajouté un bouton Undo. La fonction rattachée est similaire à la fonction traitement, mais fait l'inverse de chacune des actions, et agit sur la « self.last_letters[-1] », last_letters étant la liste des lettres sur lesquelles l'utilisateur a cliqué.

```
def undo(self):
    if len(self.last_letters)>0:
        l = self.last_letters[-1]
        self.__boutons[ord(l)-ord('A')].config(state=NORMAL)
        cpt = 0
        lettres = list(self.__motAffiche)
        for i in range(len(self.__mot)):
            if self.__mot[i]==l:
                cpt +=1
                lettres[i]= "*"

        self.__motAffiche = ''.join(lettres)

        if cpt ==0:
            self.__nbManques -= 1
            self.zoneAffichage.effacePiecePendu(self.__nbManques + 1)
        else:
            self.__lmot.config(text='Mot : ' + self.__motAffiche)
        self.last_letters.pop()
```

Pseudo

On a également modifié la fonction « nouvelle Partie » pour inclure un choix de pseudo, qu'on retrouvera ensuite dans l'historique des scores.

```
def nouvellePartie(self):
    self.__demandePseudo()

    for i in range(26):
        self.__boutons[i].config(state=NORMAL)

    self.__mot = self.__mots[randint(0,len(self.__mots) - 1)]
    self.__motAffiche = len(self.__mot)*'*'
    self.__lmot.config(text='Mot : ' + self.__motAffiche)

    self.__nbManques = 0
    self.zoneAffichage.cachePendu()
    self.last_letters = []
```

Elle fait appel à une fonction « demandePseudo », qui utilise la bibliothèque simpledialog de Tkinter.

```
def __demandePseudo(self):
    pseudo = simpledialog.askstring("Pseudo", "Entrez votre pseudo :", parent=self, initialvalue=self.__

    if pseudo:
        self.__pseudo = pseudo
    else:
        self.__pseudo = "Anonyme"
    self.__lscore.config(text=f"Joueur : {self.__pseudo}")
```

Historique des scores

On a créé un fichier « self.scores_file » qui contient le pseudo choisi par l'utilisateur, le score de chaque partie et le score total.

A la fin de chaque partie, un score est calculé et le fichier est modifié :

```
def finPartie(self, gagne):
    for b in self.__boutons:
        b.config(state=DISABLED)

    score = sum(1 for c in self.__motAffiche if c != '*') / len(self.__mot)
    with open(self.__scores_file, 'a') as f:
        f.write(f"{self.__pseudo}:{self.__mot}:{score}\n")

    if gagne :
        self.__lmot.config(text=self.__mot + ' - Bravo, vous avez gagné')
    else :
        self.__lmot.config(text='Vous avez perdu, le mot était : ' + self.__mot)
```

Un bouton « afficheHistorique » permet d'observer l'historique des parties jouées, en lisant le fichier ainsi créé, et calculant la moyenne des scores.

```python
def afficheHistorique(self):
    fenetreHist = Toplevel(self)
    texte = f"Historique de {self.__pseudo}\n\n"
    total, total_score = 0, 0
    if self.__scores_file:
        with open(self.__scores_file, 'r') as f:
            for line in f:
                parts = line.split(':')
                if len(parts) == 3 and parts[0] == self.__pseudo:
                    mot, score = parts[1], float(parts[2])
                    total += 1
                    total_score += score
                    if score == 1.0:
                        resultat = "Gagné"
                    else:
                        resultat = "Perdu"
                    texte += f"{mot} - {score} - {resultat}\n"
    if total == 0:
        texte += "Aucune partie enregistrée."
    else:
        texte += f"\nParties : {total}  |  Moyenne : {total_score/total}"
    Label(fenetreHist, text=texte, padx=10, pady=10).pack()
    Button(fenetreHist, text="Fermer", command=fenetreHist.destroy).pack(pady=5)
```

Difficulté

Enfin, nous avons fait l'ajout personnel d'une case à cocher permettant d'utiliser ou non le bouton Undo, laissant ainsi l'utilisateur choisir le niveau de difficulté du jeu.

```python
def canUndo(self):
    if self.undoButton["state"]==NORMAL:
        self.undoButton.config(state=DISABLED)
    else:
        self.undoButton.config(state=NORMAL)
```